

Tipi di variabili e tabella dei simboli, Compilazione modulare

Alessia Smeralda Brunello
Matricola: 544964

Hands-On 9

Indice

1	Introduzione	2
2	Definizione del problema	2
3	Metodologia	2
3.1	Esercizio 1	2
3.2	Esercizio 2	3
4	Presentazione dei risultati	3
4.1	Esercizio 1	3
4.1.1	Output di nm	3
4.1.2	Struttura dell'output di nm	3
4.1.3	Analisi della tabella dei simboli	4
4.1.4	Analisi con size	5
4.2	Esercizio 2	6
4.2.1	Output	6
4.2.2	Compilazione con make	6
4.2.3	Analisi dei simboli con make check	6
5	Conclusioni	7

1 Introduzione

L'obiettivo di questo laboratorio è quello di acquisire familiarità con la struttura dei programmi in linguaggio C, con particolare attenzione alla gestione della memoria e alla compilazione modulare. In particolare, vengono analizzati i simboli generati dal compilatore tramite il comando `nm` e la suddivisione della memoria nei segmenti `text`, `data` e `bss` tramite il comando `size`. Inoltre, viene introdotto l'utilizzo del tool `make` per automatizzare il processo di compilazione.

2 Definizione del problema

Il laboratorio è suddiviso in due esercizi principali:

- **Esercizio 1: Tipi di variabili e tabella dei simboli**

Analisi della tabella dei simboli di un file C compilato senza linking, con comprensione dei tipi di simbolo.

Analisi attraverso il comando `size` di come modifiche alle variabili influenzino i segmenti di memoria.

- **Esercizio 2: Compilazione modulare**

Compilazione modulare di un programma suddiviso in più file e analisi dei simboli generati nei diversi file oggetto.

3 Metodologia

3.1 Esercizio 1

Il file C fornito è stato compilato senza effettuare il linking utilizzando:

```
gcc -c ho9_1.c
```

Successivamente è stata analizzata la tabella dei simboli con:

```
nm ho9_1.o
```

Sono state poi aggiunte e rimosse variabili globali per notare le variazioni nei segmenti di memoria, compilando il programma con linking:

```
gcc ho9_1.c -o ho9_1
```

e analizzandolo tramite:

```
size --format=GNU ho9_1
```

3.2 Esercizio 2

Sono stati creati più file sorgente e un Makefile per automatizzare la compilazione. Il programma è stato compilato tramite:

```
make
```

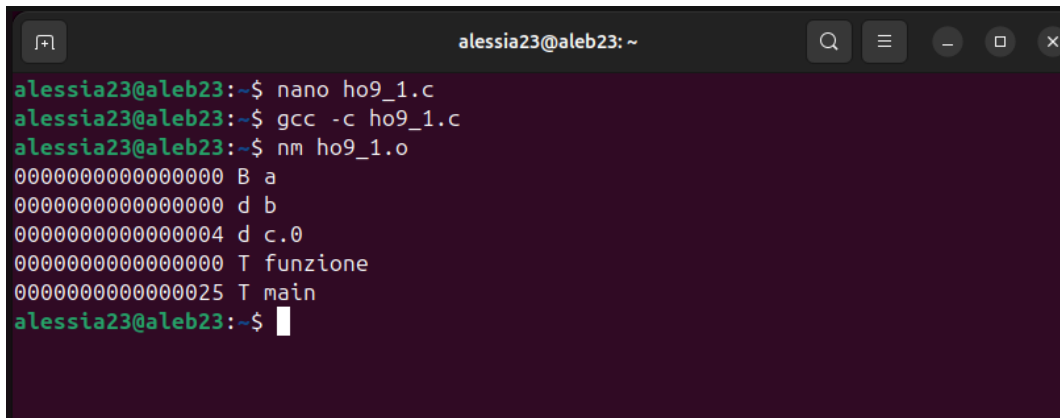
e i simboli sono stati analizzati con:

```
make check
```

4 Presentazione dei risultati

4.1 Esercizio 1

4.1.1 Output di nm



```
alessia23@aleb23: ~  
alessia23@aleb23:~$ nano ho9_1.c  
alessia23@aleb23:~$ gcc -c ho9_1.c  
alessia23@aleb23:~$ nm ho9_1.o  
0000000000000000 B a  
0000000000000000 d b  
0000000000000004 d c.0  
0000000000000000 T funzione  
0000000000000025 T main  
alessia23@aleb23:~$
```

4.1.2 Struttura dell'output di nm

- **Indirizzo:** rappresenta l'indirizzo (o offset) del simbolo all'interno del file oggetto. Può essere assente nel caso di simboli non definiti.
- **Tipo:** indica la sezione di memoria in cui il simbolo è collocato. I principali valori osservati sono:
 - T/t: simboli nel segmento **text** (codice eseguibile)
 - D/d: simboli nel segmento **data** (variabili inizializzate)
 - B/b: simboli nel segmento **bss** (variabili non inizializzate)
 - U: simboli non definiti (saranno risolti in fase di linking)
- **Nome del simbolo:** identificatore della variabile o funzione.

La distinzione tra lettere maiuscole e minuscole indica inoltre la visibilità del simbolo: le lettere minuscole rappresentano simboli locali, mentre le maiuscole simboli globali.

Tipologie di variabili:

- **globale:** dichiarate al di fuori delle funzioni e sono visibili ovunque
- **static globale:** sono dichiarate al di fuori delle funzioni con `static` e sono visibili solo nel file, l'accesso da parte di altri file è negato
- **static locale:** dichiarate all'interno di una funzione `static`, sono accessibili solo dentro la funzione in cui è dichiarata e mantengono il valore tra le chiamate

4.1.3 Analisi della tabella dei simboli

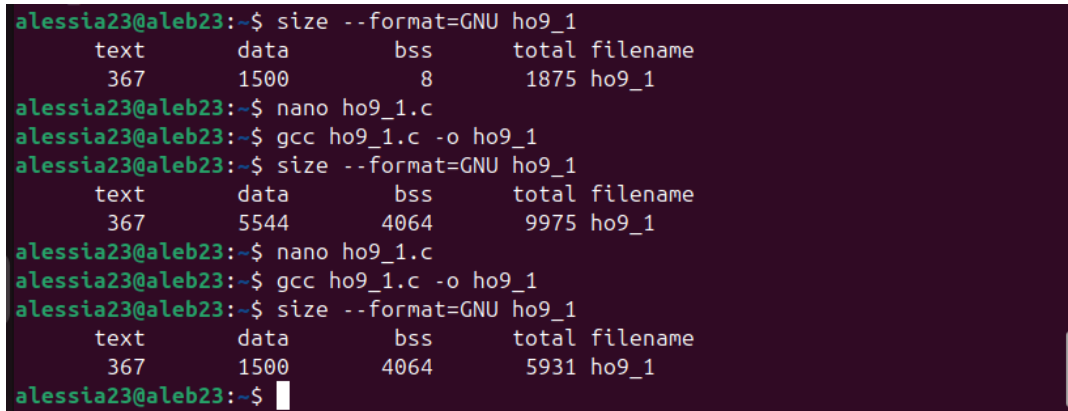
- La variabile globale non inizializzata `a` è collocata nel segmento `bss` (simbolo B).
- La variabile statica inizializzata `b` è collocata nel segmento `data` (simbolo d).
- La variabile `c`, dichiarata come statica all'interno della funzione, appare come `c.0` ed è anch'essa nel segmento `data`.
- Le funzioni `funzione` e `main` appartengono al segmento `text` (simbolo T).
- Le variabili locali, come `d`, non compaiono nella tabella dei simboli.

4.1.4 Analisi con size

Il primo output riporta i dati del programma così com'era, successivamente sono state aggiunte 2 variabili globali:

```
int big1[1000];  
int big2[1000] = {1};
```

Infine è stato eliminato big2

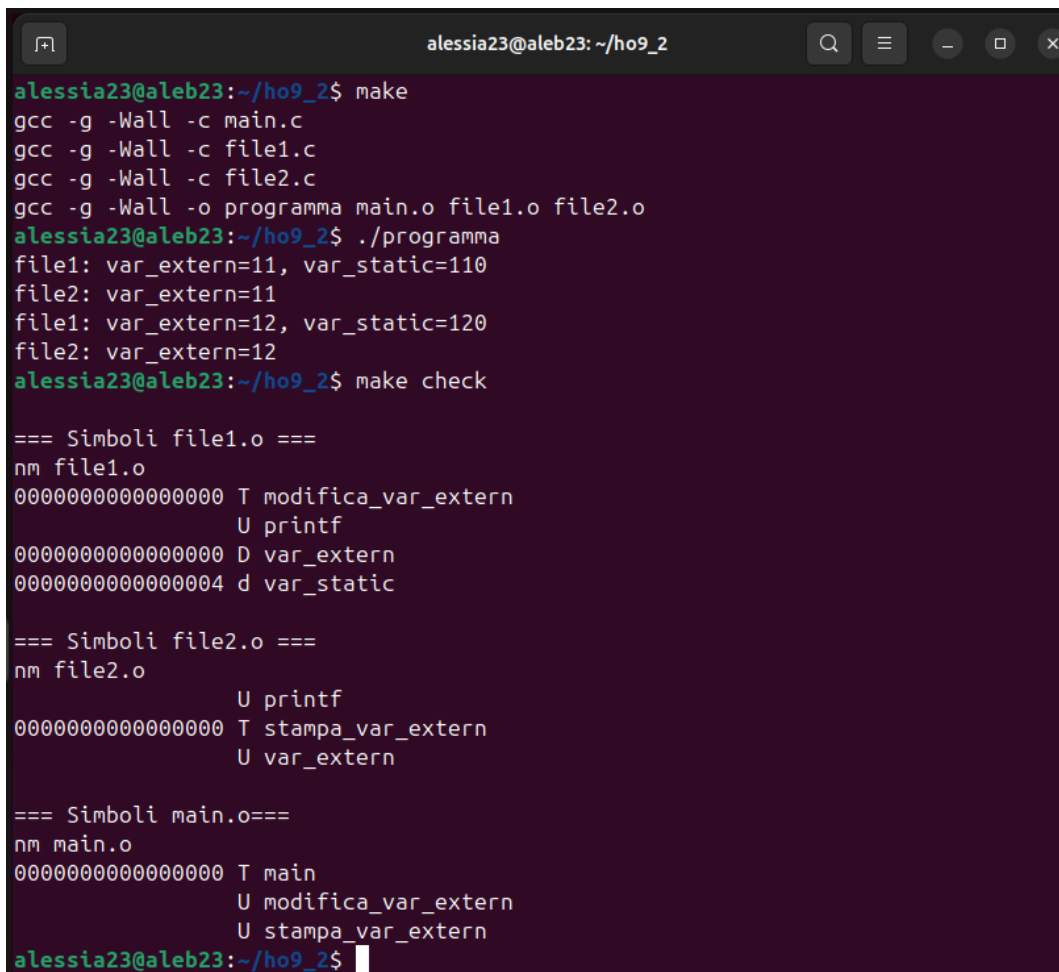


```
alessia23@aleb23:~$ size --format=GNU ho9_1  
   text    data     bss      total filename  
   367     1500        8       1875 ho9_1  
alessia23@aleb23:~$ nano ho9_1.c  
alessia23@aleb23:~$ gcc ho9_1.c -o ho9_1  
alessia23@aleb23:~$ size --format=GNU ho9_1  
   text    data     bss      total filename  
   367     5544    4064       9975 ho9_1  
alessia23@aleb23:~$ nano ho9_1.c  
alessia23@aleb23:~$ gcc ho9_1.c -o ho9_1  
alessia23@aleb23:~$ size --format=GNU ho9_1  
   text    data     bss      total filename  
   367     1500    4064       5931 ho9_1  
alessia23@aleb23:~$
```

Dalle modifiche effettuate emerge che l'aggiunta di variabili non inizializzate aumenta il segmento **bss**, mentre l'aggiunta di variabili inizializzate aumenta il segmento **data**. Il segmento **text** rimane invariato. Questo evidenzia la diversa gestione delle due tipologie di variabili da parte del compilatore.

4.2 Esercizio 2

4.2.1 Output



```
alessia23@aleb23: ~/ho9_2
alessia23@aleb23:~/ho9_2$ make
gcc -g -Wall -c main.c
gcc -g -Wall -c file1.c
gcc -g -Wall -c file2.c
gcc -g -Wall -o programma main.o file1.o file2.o
alessia23@aleb23:~/ho9_2$ ./programma
file1: var_extern=11, var_static=110
file2: var_extern=11
file1: var_extern=12, var_static=120
file2: var_extern=12
alessia23@aleb23:~/ho9_2$ make check

=== Simboli file1.o ===
nm file1.o
0000000000000000 T modifica_var_extern
                  U printf
0000000000000000 D var_extern
0000000000000004 d var_static

=== Simboli file2.o ===
nm file2.o
                  U printf
0000000000000000 T stampa_var_extern
                  U var_extern

=== Simboli main.o===
nm main.o
0000000000000000 T main
                  U modifica_var_extern
                  U stampa_var_extern
alessia23@aleb23:~/ho9_2$
```

4.2.2 Compilazione con make

Utilizzando **make** si compilano tutti i file e si crea il file **programma**, che andando ad eseguire permette di osservare che la variabile **var_extern** viene condivisa tra i file, mentre **var_static** è visibile solo all'interno di **file1.c**.

4.2.3 Analisi dei simboli con make check

Dall'analisi dei file oggetto tramite **make check** si nota che:

- In **file1.o**, **var_extern** è definita (simbolo D), mentre **var_static** è un simbolo locale (indicata con d), quindi visibile solo all'interno del file.
- In **file2.o**, **var_extern** appare come simbolo non definito (U), poiché è dichiarata ma non definita in questo file.
- In **main.o**, le funzioni **modifica_var_extern** e **stampa_var_extern** risultano come simboli non definiti (U), in quanto implementate in altri file.

Questo comportamento è dovuto al fatto che ogni file viene compilato separatamente generando un file oggetto indipendente. La risoluzione dei simboli non definiti avviene successivamente durante la fase di linking, in cui il linker collega tra loro i vari file oggetto trovando le definizioni mancanti.

5 Conclusioni

In conclusione, il laboratorio ha evidenziato il funzionamento interno della gestione dei simboli e della memoria nei programmi C.

È stata inoltre chiarita la differenza tra compilazione tradizionale e compilazione modulare, evidenziando il ruolo fondamentale del linker nella risoluzione dei simboli tra file diversi.

L'utilizzo di `make` si è dimostrato particolarmente utile per automatizzare la compilazione di progetti composti da più file, migliorando l'organizzazione e la scalabilità del codice.